

混合量化部署

由于之前我们进行yolov8模型部署时，遇到了量化精度下降的问题，这一小节我们来使用混合量化来部署模型，来减少量化精度丢失的问题。

1.模型转换

1.1 创建转换目录

打开NPU工具包的虚拟机Ubuntu20.04，创建yolov8-8.2文件夹，存放模型和量化图像等。

```
ubuntu@ubuntu2004:~$ mkdir yolov8-8.2
```

进入yolov5模型转换目录。

```
ubuntu@ubuntu2004:~$ cd yolov8-8.2/
```

创建data目录存放量化图像

```
mkdir data
```

将量化图像传入data文件夹下，例如，传入 test01.jpg 图像到 data

```
ubuntu@ubuntu2004:~/yolov8-8.2$ ls data
test01.jpg
```

在yolov5模型转换目录中创建 dataset.txt

```
ubuntu@ubuntu2004:~/yolov8-8.2$ touch dataset.txt
```

修改 dataset.txt 文件

```
ubuntu@ubuntu2004:~/yolov8-8.2$ vi dataset.txt
```

在 dataset.txt 文件中增加量化图片的路径.

```
./data/test01.jpg
```

将 yolov8n-sim.onnx 模型传入yolov8模型转换文件夹下。例如：

```
ubuntu@ubuntu2004:~/yolov8-8.2$ ls
data  dataset.txt  yolov8n-sim.onnx
```

工作目录的文件如下所示：

```
ubuntu@ubuntu2004:~/yolov8-8.2$ tree
.
├── data
│   └── test01.jpg
├── dataset.txt
└── yolov8n-sim.onnx

1 directory, 3 files
```

1.2 导入模型

```
pegasus import onnx --model yolov8n-sim.onnx --output-data yolov8n-sim.data --
output-model yolov8n-sim.json
```

导入生成两个文件，分别是 `yolov8n-sim.data` 和 `yolov8n-sim.json` 文件，两个文件是 YOLO V8 网络对应的芯原内部格式表示文件，`data` 文件储存权重，`cfg` 文件储存模型。

1.3 生成 YML 文件

YML 文件对网络的输入和输出的超参数进行描述以及配置，这些参数包括，输入输出 tensor 的形状，归一化系数 (均值, 零点), 图像格式, tensor 的输出格式, 后处理方式等等

```
pegasus generate inputmeta --model yolov8n-sim.json --input-meta-output yolov8n-
sim_inputmeta.yml
```

```
ubuntu@ubuntu2004:~/yolov8-8.2$ pegasus generate inputmeta --model yolov8n-sim.json --input-meta-output yolov8n-sim_inputmeta.yml
2023-06-19 06:13:34.277071: W tensorflow/stream_executor/platform/default/dso_loader.cc:59] could not load dynamic library 'libcudart.so.10.1'; d
error: libcudart.so.10.1: cannot open shared object file: No such file or directory; LD_LIBRARY_PATH: /home/ubuntu/Verisilicon/acuity-toolkit-binary-6.6.1/bin/acuitylib
2023-06-19 06:13:34.277199: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
0 Get binary package acuity_path [/home/ubuntu/Verisilicon/acuity-toolkit-binary-6.6.1/bin]
0 Namespace(generate'inputmeta', input_meta_output='yolov8n-sim_inputmeta.yml', model='yolov8n-sim.json', separated_database=false, which='generate')
0 Get binary package acuity_path [/home/ubuntu/Verisilicon/acuity-toolkit-binary-6.6.1/bin]
0 Load model in yolov8n-sim.json
0 Generate input meta yolov8n-sim_inputmeta.yml
-----Error(0),Warning(0)-----
```

```
pegasus generate postprocess-file --model yolov8n-sim.json --postprocess-file-
output yolov8n-sim_postprocess_file.yml
```

修改 `yolov8n-sim_inputmeta.yml` 文件中的 `scale` 参数为 `0.0039216(1/255)`，目的是对输入 tensor 进行归一化，和网络进行训练的时候是对应的。

```
vi yolov8n-sim_inputmeta.yml
```

1.4 混合量化

生成量化表文件，使用非对称量化，int8，修改 `--batch-size` 参数为你的 `dataset.txt` 里提供的图片数量。

```
pegasus quantize --model yolov8n-sim.json --model-data yolov8n-sim.data --batch-
size 1 --device CPU --with-input-meta yolov8n-sim_inputmeta.yml --rebuild --
model-quantize yolov8n-sim.quantize --quantizer asymmetric_affine --qtype int8
```

增加自定义量化层：

```
vi yolov8n-sim.quantize
```

在 `customized_quantize_layers` 下增加自定义量化层，如下所示：使用动态定点量化INT16

```
customized_quantize_layers:  
    Concat_/model.22/Concat_5_1: dynamic_fixed_point-i16
```

混合量化：

```
pegasus quantize --model yolov8n-sim.json --model-data yolov8n-sim.data --device  
CPU --with-input-meta yolov8n-sim_inputmeta.yml --hybrid --model-quantize  
yolov8n-sim.quantize --quantizer asymmetric_affine --qtype int8
```

```
D Quantize tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_243:out0) with tensor(@Concat_/model.22/Concat_5_1:out0)  
D Quantize tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_243:out0) with tensor(@Concat_/model.22/Concat_5_1:out0)  
D Quantize tensor(@Reshape_/model.22/Reshape_2_25:out0) with tensor(@Reshape_/model.22/Reshape_2_25_acuity_mark_perm_243:out0)  
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_20:out0) with tensor(@Split_/model.22/Split_21:out0)  
D Quantize tensor(@Softmax_/model.22/dfl/Softmax_18_acuity_mark_perm_240:out0) with tensor(@Reshape_/model.22/dfl/Reshape_20:out0)  
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_1_13_acuity_mark_perm_239:out0) with tensor(@Conv_/model.22/dfl/conv/Conv_17:out0)  
D Quantize tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13_acuity_mark_perm_239:out0)  
D Quantize tensor(@Slice_/model.22/Slice_12:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0)  
D Quantize tensor(@Slice_/model.22/Slice_1_15:out0) with tensor(@Reshape_/model.22/dfl/Reshape_1_13:out0)  
D Quantize tensor(@Attach_Concat_/model.22/Concat_5/out0_0:out0) with tensor(@Concat_/model.22/Concat_5_1:out0)  
I End quantization...  
I Dump net quantize tensor table to yolov8n-sim.quantize  
I Save net to yolov8n-sim.data  
I -----Error(0),Warning(0)-----  
ubuntu@ubuntu2004:~/work/yolov8$
```

注意：如果混合量化报错，可重新复制命令行的 `dynamic_fixed_point-i16` 至自定义层中的量化参数。

```
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]  
I Load model in yolov8n-sim.json  
I Load data in yolov8n-sim.data  
I Load input meta yolov8n-sim_inputmeta.yml  
I Load quantization tensor table yolov8n-sim.quantize  
W Hybrid quantization couldn't be applied for layer 'Concat_/model.22/Concat_5_1', only support the following hybrid mode:  
quantize dtype dynamic_fixed_point-i8 -> hybrid to -> float32  
quantize dtype dynamic_fixed_point-i8 -> hybrid to -> dynamic_fixed_point-i16  
quantize dtype asymmetric_affine-u8 -> hybrid to -> float32  
quantize dtype asymmetric_affine-u8 -> hybrid to -> dynamic_fixed_point-i16.  
I Start quantization...  
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]  
I [TRAINER]Quantization start...  
[TRAINER]Quantization start...  
D Optimizing network with qnt_insert_converter_layer  
Traceback (most recent call last):  
  File "pegasus.py", line 131, in <module>  
  File "pegasus.py", line 108, in main  
  File "acuitylib/app/medusa/commands.py", line 212, in execute  
  File "acuitylib/vsi_nn.py", line 592, in quantize  
  File "acuitylib/app/medusa/quantization.py", line 151, in run  
  File "acuitylib/app/medusa/quantization.py", line 50, in _run_quantization  
  File "acuitylib/app/medusa/quantization.py", line 84, in _quantize_net  
  File "acuitylib/app/medusa/quantization.py", line 103, in _generate_hybrid_table  
  File "acuitylib/optimize/optimizer.py", line 266, in apply  
  File "acuitylib/optimize/rules/quantize/hybrid_insert_converter_layer.py", line 227, in apply  
  File "acuitylib/optimize/rules/quantize/hybrid_insert_converter_layer.py", line 151, in _new_converter_layer  
  File "acuitylib/optimize/rules/quantize/hybrid_insert_converter_layer.py", line 39, in _insert_converter_layer  
UnboundLocalError: local variable 'quant_param' referenced before assignment  
[4856] Failed to execute script 'pegasus' due to unhandled exception!
```

1.5 预推理

利用前文的量化表执行预推理，得到推理 `tensor`

```
pegasus inference --model yolov8n-sim.json --model-data yolov8n-sim.data --batch-size 1 --dtype quantized --model-quantize yolov8n-sim.quantize --device CPU --with-input-meta yolov8n-sim_inputmeta.yml --postprocess-file yolov8n-sim_postprocess_file.yml
```

```
W:tensorflow:AutoGraph could not transform <acuitylib.core.execution.BackendFunction object at 0x7f795704b400>
and will run it as-is.
Please report this to the TensorFlow team. When filing the bug, set the verbosity to 10 (on Linux, `export AUTO
GRAPH_VERBOSITY=10`) and attach the full output.
Cause: Unable to locate the source code of <function BackendFunction.__call__ at 0x7f7957d850d0>. Note that fun
ctions defined in certain environments, like the interactive Python shell do not expose their source code. If t
hat is the case, you should to define them in a .py source file. If you are certain the code is graph-compatibl
e, wrap the call using @tf.autograph.experimental.do_not_convert. Original error: could not get source code
To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert
I Running 1 iterations
I Iter(0), top(5), tensor(@attach_Concat_/model.22/Concat_5/out0_0:out0) :
I 5810: 415.7803955078125
I 5811: 407.5633850097656
I 5809: 386.1991882324219
I 5812: 377.982177734375
I 5829: 371.4085693359375
I Save tensor /home/ubuntu/work/yolov8/iter_0_attach_Concat_model.22_Concat_5_out0_0_out0_1_84_2100.tensor
I Save tensor /home/ubuntu/work/yolov8/iter_0_images_238_out0_1_3_320_320.tensor
I Check const pool...
I End inference...
I -----Error(0),Warning(0)-----
ubuntu@ubuntu2004:~/work/yolov8$
```

1.6 导出模板代码与模型

输出的模型可以在 `ovxlib/yolov8n-sim_nbg_unify` 文件夹中找到 `network_binary.nb` 文件。

```
pegasus export ovxlib --model yolov8n-sim.json --model-data yolov8n-sim.data --
dtype quantized --model-quantize yolov8n-sim.quantize --batch-size 1 --save-
fused-graph --target-ide-project 'linux64' --with-input-meta yolov8n-
sim_inputmeta.yml --output-path ovxlib/yolov8n-sim/yolov8n-simprj --pack-nbg-
unify --postprocess-file yolov8n-sim_postprocessmeta.yml --optimize
"VIP9000PICO_PID0XEE" --viv-sdk ${VIV_SDK}
```

```
vx -topenvx -tcl -tvc -tval -tvalib -tvaluator -tvalproxy -t/home/ubuntu/VeriSilicon/ovxtools/ovxidebes.7.1/ovxtools/vsulator/./../common/tcl/ -t
peg.a -o gen_nbg
[Galcore]: system reserved pool0 CPU physical=0x1000000 GPU physical=0x1000000 virtual=0x1000000 size=0xf3334000
Create Neural Network: 54ms or 54899us
Verify...
Verify Graph: 6441ms or 6441203us
Start run graph [1] times...
Run the 1 time: 5.06ms or 5060.13us
vxProcessGraph execution time:
Total 5.11ms or 5109.55us
Average 5.11ms or 5109.55us
--- Top5 ---
0: 0.000000
1: 0.000000
2: 0.000000
3: 0.000000
4: 0.000000
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
D Get binary package acuity_path [/home/ubuntu/VeriSilicon/acuity-toolkit-binary-6.9.3/bin]
I Export ovxlib case with version 1.1.34
I Dump nbg input meta to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/nbg_meta.json
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_yolov8nsimprj.c
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_yolov8nsimprj.h
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_post_process.c
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_post_process.h
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_pre_process.c
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_pre_process.h
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/vnn_global.h
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/main.c
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/BUILD
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/yolov8nsimprj.vcxproj
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/yolov8nsimprj.2012.vcxproj
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify/makefile.linux
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify.cproject
I Save vx network source file to /home/ubuntu/work/yolov8/ovxlib/yolov8n-sim_nbg_unify.project
I Dump net to ovxlib/yolov8n-sim/yolov8n-simprj_fused.json
I End exporting ovxlib case...
W -----Error(0),Warning(2)-----
ubuntu@ubuntu2004:~/work/yolov8$
```

将生成的 `network_binary.nb` 文件拷贝出来备用。

2.端侧推理

4.1 编译测试程序

拷贝配套源码中的 `yolov8-hybrid-npu-code/yolov8-hybrid.tar.gz` 至SDK中 `tina-v853-100ask/package/allwinner` 目录下。解压压缩包：

```
tar -xzf yolov8-hybrid.tar.gz
```

解压后可得到端侧部署源码。

回到tina-v853-100ask目录下，重新激活环境：

```
ubuntu@ubuntu1804:tina-v853-100ask$ source build/envsetup.sh
...
ubuntu@ubuntu1804:tina-v853-100ask$ lunch

You're building on Linux

Lunch menu... pick a combo:
    1  v853-100ask-tina
    2  v853-vision-tina
which would you like? [Default v853-100ask]: 1
...
```

进入Tina配置界面

```
make menuconfig
```

进入Allwinner目录，选中yolov8-hybrid后即可编译端侧部署程序。

保存并退出Tina配置界面。

编译Tina SDK镜像，编译完成后打包生成镜像

```
ubuntu@ubuntu1804:tina-v853-100ask$ make
...
ubuntu@ubuntu1804:tina-v853-100ask$ pack
```

如果想单独编译resnet50端侧部署源码可执行：

```
make package/allwinner/yolov8-hybrid/install
```

编译生成的可执行程序在：`~/tina-v853-100ask/out/v853-100ask/compile_dir/target/yolov8-hybrid/yolov8-hybrid`

编译完成后可使用adb将可执行程序传输至开发板端：

```
adb push ~/tina-v853-100ask/out/v853-100ask/compile_dir/target/yolov8-hybrid/yolov8-hybrid /mnt/UDISK
```

4.2 运行测试程序

提前将yolov8-hybrid可执行程序、network_binary.nb模型、测试图像(320*320pix)传入开发板端，在开发板执行：

```
./yolov8-hybrid ./network_binary.nb ./test.jpg
```

```
root@TinaLinux:/mnt/UDISK# ./yolov8-hybrid ./network_binary.nb ./bus_320-320.
jpg
[19901.597939] npu[873][873] vipcore, device init..
[19901.603232] set_vip_power_clk ON
[19901.608938] enter aw vip mem alloc size 52428800 vip_heap
[19901.741590] aw_vip_mem_alloc vir 0xe13fb000, phy 0x48800000
[19901.747956] npu[873][873] gckvip_drv_init kernel logical phy address=0x48800000 virtual =0xe13fb000
Initialize vip lite success
Initialize network items success
Create Neural Network: 5.13ms or 5127.58us
Create the neural network success
buf_param.quant_forma = VIP_BUFFER_QUANTIZE_DYNAMIC_FIXED_POINT !!
Pre process the input/output data success
Start run graph [1] times...
Run the 1 time: 23.17ms or 23168.54us
vip run network execution time:
Total 23.27ms or 23268.00us
Average 23.27ms or 23268.00us
Run the neural network success
data_format=5 buff_size=352800
0: 77%, [ 25, 92, 102, 320], person
0: 36%, [ 98, 95, 168, 305], person
Post process output data success[19901.923532] npu[873][873] gckvip_drv_exit, aw_vip_mem_free
[19901.932894] aw_vip_mem_free vir 0xe13fb000, phy 0x48800000
[19901.943924] npu[873][873] vipcore, device un-init..
root@TinaLinux:/mnt/UDISK#
```

运行程序之后会保存输出图像 yolov8_out.jpg。